

# Regular Void (Revision 1)

ISO/IEC JTC1 SC22 WG21

Document Number: P0146R1

Audience: Evolution Working Group

Matt Calabrese ([metaprogrammingtheworld@gmail.com](mailto:metaprogrammingtheworld@gmail.com))

2016-02-11

## Abstract

This paper evaluates the current specification of `void` in C++ as an incomplete type with exceptional rules and proposes changes that would update `void` to instead be a complete, object type. The intent of this modification is to eliminate special casing in generic code where `void` may appear as a dependent type, such as the existing `std::promise<void>` specialization in the standard library, and to do so in a way that minimizes breakage of existing code. This also has the positive side-effect of simplifying the standard as a whole, due to the amendment or outright removal of several clauses that currently need to treat `void` separately and explicitly.

## Motivation

One of the many powerful aspects of C++ is its ability to represent generic functions and types as exemplified by the algorithms and containers of the standard library, which are able to operate on built-in and user-defined types equivalently provided that the types involved model the necessary concepts. One of the underlying properties of the language that makes this feasible is the potential for user-defined types to behave in both a syntactically and semantically similar manner to built-in types. This is because the language gives built-in object types and user-defined types value semantics, and allows for overloading and generic functions by way of templates. In practice this usually works well, however, `void` types often pose difficulty in generic code because they are incomplete and not instantiable, even though they can appear as the result type of a function that may be invoked in a generic setting. They are very unique with respect to built-in types, let alone user-defined ones.

Consider the following example, based on real-world code:

```
// Invoke a Callable, logging its arguments and return value.
// Requires an exact match of Callable&&'s pseudo function type and R(P...).
template<class R, class... P, class Callable>
R invoke_and_log(callable_log<R(P...)>& log, Callable&& callable,
    std::add_rvalue_reference_t<P>... args) {
    log.log_arguments(args...);
    R result = std::invoke(std::forward<Callable>(callable),
        std::forward<P>(args)...);
    log.log_result(result);
    return result;
}
```

The above code is simple enough, however, it breaks down in one, very important case -- when the return type of the `Callable` happens to be `void` (rvalue reference return types are also not handled here such that we are able to take advantage of the named return value optimization). In the `void` case, we'd still expect `callable` to be invoked and to have its arguments logged. Perhaps more subtle, the call to `log.log_result(result)` is meaningful even if serialization of a `void` were to end up being a no-op, since the invocation of the function still signals a successful execution of `callable` (i.e. if we get there, `std::invoke` did not propagate an exception). Ideally, all of this code would just work as written and without further specialization or indirection to account for the case where `callable` returns `void`.

## Example in the Standard

The motivation for this change does not exist solely in user-written generic code, as symptoms of the underlying problem are visible even in the standard library itself. Most clearly, this can be seen with the need for specializations of `std::promise` and `std::future` for the `void` type. It can further be seen in proposals such as the `void` specialization for the expected template<sup>[1]</sup>. Unfortunately, even though we already have specializations for `std::promise` that at least allow `std::promise<void>` to be created, dealing with that type in generic code by users is still difficult due to the way that `void` is handled in the language and the fact that the specializations provide no common interface for setting the promised value (this latter point does not actually have to be the case even without changes to the language itself, though such an interface would be less than ideal). An example of where things break down in simple generic code that deals with `std::promise` can be seen here:

```
template<class T, class Fun>
void set_value_with_result(std::promise<T>& p, Fun fun) {
    p.set_value(fun());
}
```

In the above code, if `T` happens to be `void`, our code will fail because `set_value` in that case takes no parameters other than the promise itself, and you also cannot pass a `void` expression as a function argument. If you want to properly handle `void`, you'd need an overload:

```
template<class Fun>
void set_value_with_result(std::promise<void>& p, Fun fun) {
    fun();
    p.set_value();
}
```

In this simple case the overload is fairly trivial, but in practice, such functions can be sufficiently more complicated and it can be difficult to provide such overloads in a way that is not prone to error or full of explicit compile-time branching via template metaprogramming, or the proposed `constexpr_if`. In an ideal world, no overload nor branching would be required. This proposal makes that a reality.

*Note: This paper strictly addresses language changes to `void` without proposing significant changes to the standard library. For changes to the standard library, such as the removal of the `void` specializations of `std::promise`, see the proposal drafts "Remove Future-Related Explicit Specializations for Void" [\[2\]](#) and "Standard Library Support for Void" [\[3\]](#), which both depend on the viability and acceptance of this proposal.*

## Simplification and Teachability

Apart from allowing the above motivating cases to be implemented and work as-is, the change of `void` to be a Regular [\[4\]](#) type also simplifies C++ itself by making the type system more consistent. Specifications in the standard that account for `void` change from being made up almost entirely of special cases, to being governed by the existing rules of object types, which is what `void` becomes as a part of this proposal. This makes it easier for users to think of `void`, as it allows them to think of it in much the same way that they do most other types in the language. It is akin to thinking of  $\emptyset$  as a number, which it is.

## Problem Description

Generic programming in C++ is especially powerful because with care, concept requirements can usually be specified in a way such that they may be met nonintrusively with respect to an existing type, making it possible to have the type model a concept even if a programmer otherwise has no control over its implementation (i.e. it is a part of a dependency or is a built-in type). However, sometimes requirements are specified that simply cannot be met nonintrusively. A good example of this is when a type requirement is the instantiability of that type, or proper semantics regarding special member functions. In these cases, if the type does not already meet these requirements, they cannot be met without modifying the type in question. If the type in question is a part of a dependency or happens to be a fundamental type of the language, then that type cannot be made to model the given concept without either patching the dependency or changing the language, respectively. This is particularly unfortunate if a type *could* model the concept without any cost, yet simply does not do so.

The problem that generic libraries face when dealing with `void` can be thought of as precisely this last case. As currently specified in the language, `void` cannot be instantiated, nor can it be copied or compared, even though it can be logically thought of as a monostate type.

## Recommended Solution

Briefly, the solution that is recommended by this proposal is to change `void` to a complete object type and to do so in a way that prevents considerable breakage to existing code. Although this change may sound risky, the alterations are specified in such a way that potential breakage is limited. Descriptions of the few potential breaking cases are detailed later.

In reality, what is likely if these changes were to be made, is that most existing users of the language likely would not even notice the update had they not heard about it before-hand, and their code would not be affected adversely by such ignorance. What is proposed more formally below is a fairly straight-forward simplification to the standard, removing many parts that currently need to explicitly mention `void` types in wording due to their nature as a unique kind of type in C++. The most risky of changes are discussed in this document explicitly, separate from the actual changes that are suggested for the standard.

## Existing Practice in Other Languages

The notion of `void` as an instatiable type is nothing particularly new to programming languages as a whole. Indeed, descriptions of `void` types at a more general level than C++ present `void` as a unit type, much as it is proposed here. That C and C++ have a `void` type that is not instantiable is incidental to its original use, but the limitations have surfaced over time. Separate from C and C++, at least one modern, mainstream language has a `void` type that is a Regular unit type – the Swift programming language's `void` type, called `Void`, is both instantiable and an alias of the language's empty tuple type. [\[5\]](#) Apart from Swift, several other language provide such a void-like unit type, including Rust [\[6\]](#) and Haskell [\[7\]](#) via `()`, and Python [\[8\]](#) via `NoneType`.

## Overview

### Thinking about Void

Presented below is a `struct` definition that is analogous to what is proposed for `void` in this paper. The actual definition is not a `class` type, but this serves as a fairly accurate approximation of what is proposed and how developers can think about `void`. What should be noticed is that this can be thought of as adding functionality to the existing `void` type, much like adding a special member function to any other existing type that didn't have it before, such as adding a move constructor to a previously non-copyable type. This comparison is not entirely analogous because `void` is currently no ordinary type, but it is a reasonable, informal description, with details covered later.

```
struct void {
    void() = default;
    void(const void&) = default;
    void& operator =(const void&) = default;

    template <class T>
    explicit constexpr void(T&&) noexcept {}
};

constexpr bool operator ==(void, void) noexcept { return true; }
constexpr bool operator !=(void, void) noexcept { return false; }
constexpr bool operator <(void, void) noexcept { return false; }
constexpr bool operator <=(void, void) noexcept { return true; }
constexpr bool operator >=(void, void) noexcept { return true; }
constexpr bool operator >(void, void) noexcept { return false; }
```

Basic Changes

The following is a high-level overview of the proposed changes along with comparisons between scalar types, the proposed `void` type, and the existing `void` type. Consistencies with scalar types are highlighted in green, inconsistencies are highlighted in red, and situations where the `void` type does *more*, but in a compatible manner, are highlighted in orange. Rationale for exactly the changes that are described here is provided in future sections.

Type Changes

|                                        | Scalar Type             | Proposed void               | Current void                            |
|----------------------------------------|-------------------------|-----------------------------|-----------------------------------------|
| Classification                         | Object Type             | Object Type                 | Not an Object Type                      |
| Completeness                           | Complete                | Complete                    | Incomplete                              |
| Copyability and Assignability          | Copyable and Assignable | Copyable and Assignable     | N/A                                     |
| Equality Operators                     | Yes                     | Yes                         | N/A                                     |
| Relational Operators                   | Yes                     | Yes                         | N/A                                     |
| Arrays of $\tau$                       | Allowed                 | Allowed                     | Not Allowed                             |
| References of $\tau$                   | Allowed                 | Allowed                     | Not Allowed                             |
| Dereference of $\tau^*$                | Allowed                 | Allowed                     | Not Allowed                             |
| Arithmetic with $\tau^*$               | Allowed                 | Allowed                     | Not Allowed (GCC Permits)               |
| Deletion via $\tau^*$                  | Allowed                 | Allowed                     | Undefined Behavior (GCC and Clang Warn) |
| Non-Type Template Parameters of $\tau$ | Allowed                 | Allowed                     | N/A                                     |
| Lack of Explicit Return of $\tau$      | Undefined Behavior      | Allowed (Value-Initialized) | Allowed                                 |

Function Type Changes (Dependent  $\tau$ )

|                                       | Scalar Type              | Proposed void            | Current void                    |
|---------------------------------------|--------------------------|--------------------------|---------------------------------|
| Type of <code>int foo(T)</code>       | <code>int(T)</code>      | <code>int(T)</code>      | Invalid or Substitution Failure |
| Type of <code>int foo(const T)</code> | <code>int(T)</code>      | <code>int(T)</code>      | Invalid or Substitution Failure |
| Type of <code>int foo(T name)</code>  | <code>int(T)</code>      | <code>int(T)</code>      | Invalid or Substitution Failure |
| Type of <code>int foo(int, T)</code>  | <code>int(int, T)</code> | <code>int(int, T)</code> | Invalid or Substitution Failure |
| Type of <code>int foo(T, int)</code>  | <code>int(T, int)</code> | <code>int(T, int)</code> | Invalid or Substitution Failure |

Function Type Changes (Non-Dependent  $\tau$ )

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|                                       | Scalar Type              | Proposed void                                      | Current void       |
|---------------------------------------|--------------------------|----------------------------------------------------|--------------------|
| Type of <code>int foo(T)</code>       | <code>int(T)</code>      | <code>int()</code> <a href="#">(See Rationale)</a> | <code>int()</code> |
| Type of <code>int foo(const T)</code> | <code>int(T)</code>      | <code>int(T)</code>                                | Invalid            |
| Type of <code>int foo(T name)</code>  | <code>int(T)</code>      | <code>int(T)</code>                                | Invalid            |
| Type of <code>int foo(int, T)</code>  | <code>int(int, T)</code> | <code>int(int, T)</code>                           | Invalid            |
| Type of <code>int foo(T, int)</code>  | <code>int(T, int)</code> | <code>int(T, int)</code>                           | Invalid            |

## Considerations

### Unary Functions Taking Void

As `void` becomes a Regular type, it is essential that it is allowed as a function parameter type, just like any other object type in the language. Without that it is not suitable in generic code, as seen in the `logging` and `std::promise` examples. This, however, brings about one minor subtlety. Syntactically, in current C++, a type such as `int(void)` is valid yet is not a function taking `void` as a parameter. Instead, it is a function taking no parameters. In other words, even though it is syntactically different it specifies the same type as `int()`. This is obvious, but it means that we would not be able to reclaim that syntax without causing a large amount of breaks to existing code.

The suggested solution to this problem is to not reclaim this syntax, at least not at this time, though only when the `void` type is not a dependent type. Importantly, we can and should use this syntax when `void` is dependent, as is explicitly suggested in this proposal, and is thankfully not allowed as a means to specify a nullary function in the present state of the language. Because of that, it is very unlikely (though still possible in hypothetical, contrived cases mentioned later) that such a change would break existing code. The reason why it is important that we allow this syntax in a dependent context is because this is precisely where it matters to generic code. By utilizing the syntax in this context, we have little-to-no chance of breaking any existing code while also allowing for generic code to be written in a way that does not need to specialize or go through indirection in the event that `void` happens to show up as a dependent type.

This leaves us with one question, though. If, when in a non-dependent context, you cannot declare a unary function that takes a `void` parameter using this syntax, how exactly would you specify such a function type? There are multiple options, but ultimately it is proposed here that users either specify the `void` parameter type with cv-qualification, which is not valid in this context in current C++, or alternatively they can simply give the parameter a name, which is also not currently allowed. Both of these solutions are specified as a part of this proposal. Rationale for why this seemingly subtle syntax is the preferred solution over alternatives is presented later.

Finally, it deserves to be repeated that this quirkiness is only required when the function in question takes *exactly* one parameter, where that parameter is `void`, and where that `void` is not a dependent type. If the function type takes multiple parameters, then the user needs to do nothing special. Similarly, if the `void` type is dependent, the user also needs to do nothing special. This only would affect a small amount of yet-to-be-written code that only a certain class of users would ever want to write, and it does not at all negatively affect the simplicity of writing generic code.

### Substitution Failure Is Not an Error

While mostly theoretical, there are potentially SFINAE (Substitution Failure Is Not an Error) exploits that might be in use that could take advantage of `void`'s properties, such as its inability to be used as a function parameter type in a dependent context, and its inability to be used in a `sizeof` expression. At the very least, such exploits are not known to have been used in standard library implementations nor have they been encouraged in Boost. It is suspected that such properties are exploited rarely, if it all, and in those few cases if and where they are exploited, there already exist drop-in alternatives via other forms of SFINAE exploits. Note that the `sizeof` trick that is or was frequently used when defining metafunction traits is distinct from what is described here and would not be affected. Similarly, `std::enable_if` is not affected by this.

## Alternatives

The solution that is proposed was arrived at after carefully considering possible alternatives. Notable suggestions from other parties are presented below.

### Library-Based Solutions

As described in the motivation section, the current way that generic code deals with `void` is by special-casing, often through helper types and templates. It has been suggested that we possibly consider standardizing helper templates that ease dealing with `void` in generic code. The problems with this are that such facilities are both complicated to specify and require people writing generic code to actually know about and use those facilities. When and why to use them is subtle. Readers of that code

who are not familiar with the problem are also baffled by the subtleties. By simply making `void` an object type, we solve our problems completely and in a way that is easy for users to utilize, and also easier to specify fully and properly in the standard when compared to a complicated set of library facilities.

To stress this, as a generic programmer I have used a variety of library-based techniques over the years to help deal with `void`. The results of such techniques, while "better" than handling each case individually, are still not for the faint of heart due to the dependence on advanced C++ techniques and the fact that those techniques need to be explicitly employed by the developer of the generic code. A brief outline of the most successful of those techniques, in my experience, revolves around the following methodology:

1. Introduce a `Void` type that is just a `Regular` unit type.
2. Introduce metafunctions and high-order functions that are already special-cased for `void`.
  - A metafunction that is an identity metafunction for all types except `cv-void`, in which case it converts to `cv-Void`
  - The inverse of the above metafunction
  - A version of `std::invoke` that returns `Void` instead of `void` when invoking a `Callable` that returns `void`
  - A high-order function that takes a unary or nullary function "F" and a separate function "I" to invoke, then invokes "F" with the result of "I", unless "I" returns `void` or "F" is only able to be invoked with no arguments, in which case, "I" is invoked followed by "F" being invoked with no arguments.
  - *Other, more obscure high-order functions*
3. Write the bulk of the internals of generic code as normal -- do not try to account for `void`.
4. At the interfaces between the generic code and the user, use the aforementioned metafunctions and high-order functions to avoid explicit branching and top-level specialization.
5. Continue special-casing in the remaining places where the library-facilities aren't enough.

As was warned, the technique is not pretty, and certainly not helpful to average Joe programmer. While it successfully removes redundancy, most specialization in high-level code, and other types of errors, it makes interfaces of the code pretty much unreadable. It also requires an understanding of advanced techniques in C++. Because we want/need to isolate the user from this helper `void` type, the interfaces that we produce are just as difficult to use from other generic code as existing interfaces (such as `set_value` of `std::promise<void>`) unless we also provide `Void`-aware interfaces.

No one wants to write or read code like this and we shouldn't encourage it if there is a proper, language-level solution.

## Void as *Almost* an Object

The idea here is that instead of making `void` an object type, we instead just make it act a little bit like one in certain contexts. The proclaimed rationale for this approach is that we introduce fewer changes to the meaning of `void` so as to minimize breakages and to not make `void` do more than is necessary. For instance, it was suggested that we make it possible to create an instance of `void` on the stack, but not be able to pass it to a function as an argument, or make an array or tuple that contains a `void` element. While the intention here is in the right place, the rationale is flawed in several ways:

- By just making `void` *almost* an object, we'd only be trading some special-casing in generic code for other special-casing.
- The changes to the standard that would make `void` an object are simpler to specify than trying to make `void` *almost* an object type with special rules.
- All of the functionality of a `Regular` type really is beneficial, since function result types are often worked with in ways that have such constraints in generic code.
- No real world examples have been provided where such a specification would at all reduce breakage to existing code when compared to making the full change that is proposed in this paper. Making `void` *almost* an object could, however, scatter the complete changes across multiple standards, leaving the language with unnecessarily subtle differences between iterations with no discernible benefit.

## An Entirely New Void Type

Also suggested was the possibility of introducing a new type to the language, separate from `void`, that behaves in the way that this proposal specifies the existing `void` type should behave. The idea of this solution is that people who want such a monostate type can opt-in for it, leaving the existing `void` type unchanged. This initially may seem like a reasonable solution, but it fails to actually solve the generic programming problem since *users* of the generic code would be the ones who would be required to now take advantage of that new type. Library developers would still need to special-case for `void` in practice because of this.

## Explicit Void

As described earlier, this proposal recommends using `cv-void` or a named `void` parameter for representing a unary function with a `void` parameter when `void` is not dependent. One alternative to using `cv-void` or a named `void` parameter for representing a unary function having a `void` parameter is to instead propose an entirely new syntax for that case. For instance, one possibility is to allow a declaration such as:

```
void foo(explicit void) {}
```

At first this may seem like a good idea since it makes the difference between it and the current meaning of a `void` parameter list more drastic, however it has a few drawbacks:

- It introduces a syntax that would only work for `void` (unless it were also proposed to work for other types for the sake of consistency).
- It is a more complex change as it requires altering the language grammar.
- It introduces a syntax that may be deprecated in the future.

The last point is a bit subtle, though it may be the most important. Ultimately, in a future standard it would be a simplification to the language to make a parameter-list consisting of a single, unnamed, cv-unqualified `void` parameter behave no different from most other object types. That is to say, that syntax would eventually be used to declare a unary function taking a parameter of type `void`. It is not recommended that we update the meaning of that syntax immediately as a part of this proposal because that would change the meaning of a large body of existing code in a way that would cause breakages, including sometimes in ways that only change behavior of existing programs at runtime. If, in the meantime, we were to introduce an entirely new syntax and alter the language grammar solely to allow the declaration of a parameter of type `void`, we'd likely want to eventually deprecate and remove *that* syntax once the desired syntax is ripe to be used. On the other hand, if in the time being we only use cv-qualified `void` or a named `void` parameter to represent an actual `void` parameter, then if/when the current meaning of `void` is to finally be replaced, any code that declared a single parameter of type `void` in the currently proposed manner would not need to be updated, since even then it would remain as a valid way to declare such a parameter (just as it is a valid way in the language right now to declare parameters of non-void type).

So, at the cost of being a little bit subtle in newly-written code, we avoid cluttering the language with new syntax and also avoid having to remove a special syntax at some point in the future. It is a simpler change in the language to make right now, in addition to being better in the long term.

## Partial-Alias of Void

Similar to the idea of creating a separate `void` type is the idea of introducing a "partial alias" for `void` that behaves in the manner suggested by this proposal, but is otherwise treated as an alias of `void` in all or most other ways (template specializations on `void` and the new alias would be the same, for example). The rationale given for this is that it avoids changing the meaning of `void`, but may provide a route toward solving some problems that have been presented. This would be a very unique addition to the language that makes the standard more complicated and difficult to learn. It also has odd implications and raises some questions. Most importantly, if dependent `void` does not behave in the manner proposed by this paper, then the motivating cases of this proposal are not actually covered. If dependent `void` *does* behave in a manner consistent with this paper, there will still be a change in meaning for `void` when the type is dependent, which is what the "partial alias" suggestion is trying to avoid. At that point, the "partial alias" option is in many ways equivalent to the "explicit void" option, but with a new special kind of type/alias and the complexity that comes with it.

## Parameter Collapsing

In discussions, it has been suggested by multiple parties independently to consider "collapsing" `void` parameters in a function type in a way that makes a function type specified as `void (int, void)` equivalent to the function type `void (int)`. Similarly, when invoking a function and passing a `void` expression as an argument, that argument would instead be ignored and the invocation would actually be passing one less argument than it would appear. Though initially this might not look problematic, not only does it keep `void` as a special type with special rules that can really make it difficult to write certain generic code, but it also introduces subtle ambiguities or other errors. For example:

```
enum class log_option { simple, verbose };

template<class T>
void log(T what_to_log, log_option option = log_option::simple);

template<class Fun>
void log_result(Fun fun) {
    log(fun(), log_option::verbose);
}

int main() {
    // logs an int using the verbose option
    log_result([]{ return 0; });

    // If internally the first argument "collapses" because the function returns void,
    // then we are no longer logging "void" with the "verbose" option, but instead,
    // we are logging an instance of "log_option" with the "simple" option.
    log_result([]{});
}
```



As shown in the example, this feature is not logically sound even though it initially might seem to solve certain problems.

## Frequently Asked Questions

### Doesn't This Proposal Introduce *More* Special-Casing for `void`?

No. This is most easily seen by examining the [tables of changes](#) or the [specification](#). What should be noticed is that while the changes that this proposal introduces provide more consistency with existing types, they do not introduce any new inconsistencies that didn't already exist. Not only is the type system more consistent because of this, but that fact is reflected in the wording in the standard, where the vast majority of the changes are simply removals of existing special cases.

### Why Isn't `sizeof(void)` Equal to `0`?

One suggestion that has repeatedly come up is to have `sizeof(void)` report `0` and to allow multiple instances to share the same address. This would prevent users from having to use tricks akin to the empty base optimization in order to make more optimal usage of memory. Ideally, this would be the case, however such a change to the language is both vast and out of scope of the proposal. Allowing a type to have a `0` size and to allow separate instances to share an address implies drastic and subtle breaking changes to existing code. For instance, if you were to make an array of such a `void` type, a pointer, at least in the traditional sense, would no longer be able to be used as an iterator into that array (notably meaning that generic code which relies on this would now fail for such a size `0` type). As well, any code that relies on an object's type and address as unique would fail for `void` types, even though it is otherwise perfectly acceptable. Finally, if such a size were permitted for `void`, it should really be allowed for *any* type, including user-defined types. Having a special rule for `void` would make one more thing to have to think about and deal with differently for `void` types. Instead, this proposal opts to leave the size of `void` unspecified and thereby governed by existing language rules. In practice, it is expected that `void` will likely be size `1` in most implementations, though this is not required. If an eventual change were made to the language to allow for size `0` types, then `void` would be able to implicitly take advantage of that.

### Does This Break `std::enable_if`?

No. The uses of `std::enable_if` do not depend on the uniqueness of the `void` type. As described [earlier](#), while there are contrived, hypothetical SFINAE exploitations separate from `std::enable_if` that can break, they are not ones endorsed by the standard library nor are they encouraged by Boost. If such exploits turn out to be used, there are trivial replacements in the form of alternative SFINAE exploits.

### In Practice, Would This Break ABI Compatibility?

No. Because we are changing the `void` type, it might initially seem as though we'd be breaking compatibility with existing, prebuilt libraries, including C libraries. This is untrue. Our alterations do not change the meaning of existing, valid function declarations, nor do they add any state to the `void` type that needs to be communicated. The fact that functions with the return type `void` now return objects from the point of view of those who use C++ does not affect this.

### Doesn't `constexpr_if` Make Branching for `void` Easier?

With the acceptance of `constexpr_if`<sup>[9]</sup>, some have voiced that the problems this paper solves may already have an upcoming solution. The idea is that because programmers can now more easily branch off in a function template definition explicitly via `constexpr_if` based on whether or not a type is `cv void`, the changes to `void` proposed in this paper would not be necessary. Unfortunately, while `constexpr_if` does aid in turning specializations into more imperative branching, the goal of making `void` a Regular type is to make specializations and branching entirely *unnecessary*, having templates work correctly with `void` without any effort taken by the developer of that template.

### Isn't It Illogical to Support *some-operation* for `void`?

In several instances, people have asked whether or not certain operations or compound types make sense for `void` and wonder why they are allowed. These concerns usually come down to a belief that because a particular operation would effectively be a no-op, it would be illogical for such an operation to appear in code, since it would likely be a user-error. The issues with such a belief are immediately evident for those who write generic code, as it is unable to be known whether or not a logical operation actually *does* anything, and so this paper will not go into too much detail on that aspect. Some examples of functionality that people have questioned the need for with respect to `void` are as follows (not a complete list):

- Making an instance of `void`.
- Making an array of `void`.
- Making a tuple that contains a `void` element.
- Copying a `void` object.
- Passing a `void` object as a function argument.

- Forming a reference to `void`.
- Using pointer arithmetic on a pointer to `void`.
- Comparing instances of `void`.

Some of these points are already covered in the motivation section, though perhaps deserving of more explicit mention are the array, tuple, reference, and pointer arithmetic cases. These can all be rather concisely explained with a simple code example that invokes  $N$  Callables in generic code, aggregating the results into an array:

```
// Execute N functions with the same return type,
template<class ReturnType, class...Callables>
std::array<ReturnType, sizeof...(Callables)>
invoke_functions(Callables&&... callables) {
    return {{std::invoke(std::forward<Callables>(callables))...}};
}
```

In generic code, `void` can very easily show up as a dependent result type. Here, it should be clear that there is nothing illogical about producing an aggregate of function results given that there is nothing illogical about dealing with a single result. Similarly, just as with any other array in C++, one would expect a pointer to an element of an array of `void` to act as a valid iterator into the array (and this also implies pointer arithmetic), otherwise `void` would fail to work for a large body of generic code.

The above example applies to `std::tuple`, just the same, if you remove the restriction of the functions all having the same return type. Now consider the signature of the functions involved when creating an instance of that tuple type, and the function types that arise when accessing that tuple — you will find that references to `void` and `void` parameters may naturally pop up, and there is no problem with this. As specified in this proposal, `void` is simply an object type like any other object type. This type just happens to only be able to represent a unit set of values (it would be a monostate type). This would not even be the first of such types to exist in the language (see `std::nullptr_t` as one such example, with multiple others exist in various proposals).

## Doesn't This Remove the Notion of "No Result?"

No.

One concern that has been brought up is that by making `void` an object type, we'd be removing the ability in the language to represent a function with no result and be left with a language that can only represent functions with exactly one result. Further, it has been suggested that we should go in the other direction, and instead of removing the current meaning of `void`, we should directly support the notion of a function with multiple results, including no results, and to have `void` represent such a lack of result. Though this may sound sensible at first, it actually ends up being an entirely orthogonal concern that is not in conflict with the changes to `void` that are proposed here.

Before going further into the details, it should first be stressed that functions in C++ can already represent any number of results, including what many refer to as "none," by way of containers, tuples, and ranges, just as functions in mathematics have the ability to represent such results even though, strictly speaking, the application of a function in mathematics maps to one result. A language-level multiple return value facility would only alter the way in which users can opt to *convey* multiple results and how such results are operated on syntactically. Such a facility may prove to be valuable for the language, as it might allow for more opportunities to elide copies and moves of objects that are currently unable to be elided, it may make it easier to expand out results as separate arguments to function calls in a single expression, and it may be able to reduce compile times when compared to existing alternatives. Again, though, these benefits are not at all in conflict with the contents of this proposal.

Though this is all hypothetical as there is no current proposal for actually providing the language with a multiple return value facility, we can reason about the desirable properties of such a facility if one were formally proposed. As one quickly sees, with such a facility, there are still very compelling reasons to allow a representation of multiple results as a single object that can be passed around and manipulated as a whole, much like one does with a `std::tuple`, and just as is proposed in this paper regarding `void` (note, too, that in C++, a `std::tuple<>` with no template arguments is a `Regular` type, and this is extremely important for generic code). For example, imagine that you wish to create a `std::future` that corresponds to the result of some nullary function `func`. In current C++, this type may be specified as `std::future<decltype(func())>`. Notably, this works in current C++ even for a `void` return type. If the language had a way to directly represent a function with  $N$  different return values, what would a user expect from such a `std::future` instantiation? If the result of the function call is not implicitly dealt with as a single entity, regardless of the value of  $N$ , but instead is dealt with as something more akin to a variadic list of return types, that bit of code either wouldn't compile, or, if expanded, would pass  $N$  different type arguments to `std::future` (and the 0, or `void` case, would now fail, unless we were to maintain an inconsistency for such a return type).

If the result type were not treated as a single entity, in order to make such a `std::future` instantiation work for all  $N$  in this case the user would likely just always manually bind the results together into a tuple before forming the `std::future` template argument. A generic high-order function that needs to create such a facility would likely now always bind the result of the call to a tuple. Unfortunately, this also affects usage of the result when retrieved from the `std::future`. In other words, while retrieving the results from the function directly, the user would get some kind of multiple-return-value construct, retrieving the results from the `std::future` would yield a tuple type. The developer would now have to expand out that tuple in order to use it in places



that the result of the original function would be usable.

Perhaps, alternatively, this hypothetical multiple return value proposal that doesn't represent the multiple results as a single entity would also introduce changes to templates, such as `std::future`, to make the template parameter list variadic. Then, users could expand their result type as  $N$  template arguments and the return value of `get()` could directly result in the language-level multiple return value facility. Not only would this make things much more complicated at the library level, but `std::future` is not at all special here. Any time you instantiate a template with the result type of a function, this issue would show up, and the set of templates that you may instantiate in this manner is potentially any template that has a type-template-parameter.

On the other hand, if you directly represent the multiple return value facility as a single object, dealing with such an entity is consistent with existing return types and has no negative implications regarding generic code. What you'd actually have, in this case, is a language-level tuple type, and just like with `std::tuple`, a tuple that contains no elements is still, itself a `Regular` type, which is important for generic code.

So, briefly, even if we were to have direct support for multiple return values in the language, it would be beneficial for such a feature to be synonymous with direct, language-level tuple support in that the return value itself could be directly used as a single object. *Not* treating such a facility in this manner would actually make generic code more difficult to write correctly as opposed to less difficult. Further, just as is the case with `std::tuple<>`, such an "empty" set of results should be able to be dealt with like any other case and should be a `Regular` type. In the presence of such a multiple return value facility using some intrinsic tuple type, the `void` type could even hypothetically be specified to be an alias of such an empty tuple, and so an assertion that the `void` type of this proposal would be inconsistent with the `void` type in the presence of a hypothetical multiple return value facility is incorrect (in fact, this is exactly what the Swift programming language does). In either case, having a `Regular void` type is just as compelling.

## Isn't This a Change to the *Meaning* of `void`?

No? Yes? It depends on how you personally choose to think about `void` in current C++

. This ideological concern is both subjective and irrelevant to how code is written and to what values can be represented in a C++ program. While it is true that the mental model some developers have may change, this would not at all adversely affect their code, and on the contrary, this would make it easier to correctly write high-order functions that encounter `void` as a dependent type. While a particularly programmer can maintain a world view where `void` is a totally different "kind" of type, as the current standard represents it, this is a more complicated world view than the alternative. It is analogous in mathematics to treating  $\emptyset$  as something other than a number. You can have a world view such as that, as people did for many years, but it certainly does not make anything easier to reason about.

Taking a step back, we should not let raw intuitions guide our view of language design or of abstraction in general, whether in programming or in other disciplines. Rather, we should lift our abstractions from introspection of the world and of the solutions that we discover. At that point, if there is inconsistency between reality and our intuitions, then our *intuitions* are what are in error. It only hinders progress to deny that.

## Proposed Solution

The `void` type is updated to be an object type with care taken to minimize breaking changes to existing code. This implies a considerable number of changes to the standard, although most of those changes are simplifications due to the removal of redundant statements mentioning `void`, which are now implicitly covered by the fact that `void` is a complete object type that is considered a scalar. Additional updates are planned for the LEWG<sup>[2][3]</sup>, but they are outside of the scope of this proposal and are predicated on the viability of the language changes presented here.

The following differences are with respect to the working draft N4567<sup>[10]</sup>, which is the most recent working draft for C++ at the time of writing this revision.

Change in **§3.9 Types [basic.types]** paragraph 5 (`void` is now a complete type):

A class that has been declared but not defined, an enumeration type in certain contexts (7.2), or an array of unknown size or of incomplete element type, is an *incompletely-defined object type*. Incompletely-defined object types ~~and cv-void~~ are *incomplete types* (3.9.1). Objects shall not be defined to have an incomplete type.

Change in **§3.9 Types [basic.types]** paragraph 8 (`void` is now an object type):

An *object type* is a (possibly cv-qualified) type that is not a function type~~7~~ and not a reference type~~, and not cv-void~~.

Change in the first part of §3.9 Types [basic.types] paragraph 9 (void is now a scalar type):

Arithmetic types (3.9.1), enumeration types, `void`, pointer types, pointer to member types (3.9.2), `std::nullptr_t`, and cv-qualified versions of these types (3.9.3) are collectively called *scalar types*. ...

Remove §3.9 Types [basic.types] paragraph 10 (remove redundancy now that void is a scalar type):

A type is a literal type if it is:

- ~~— possibly cv-qualified void; or~~
- a scalar type; or
- a reference type; or
- an array of literal type; or
- a possibly cv-qualified class type (Clause 9) that has all of the following properties:
  - it has a trivial destructor,
  - it is an aggregate type (8.5.1) or has at least one `constexpr` constructor or constructor template (possibly inherited (7.3.3) from a base class) that is not a copy or move constructor, and
  - all of its non-static data members and base classes are of non-volatile literal types.

Change in §3.9.1 Fundamental types [basic.fundamental] paragraph 9 (redefine void to be a monostate type):

A type `cv void` ~~is an incomplete type that cannot be completed; such a type has an empty set of values~~ *has only one value*. ~~It is used as the return type for functions that do not return a value. Any expression can be explicitly converted to type `cv void` (5.4). An expression of type `cv void` shall be used only as an expression statement (6.2), as an operand of a comma expression (5.19), as a second or third operand of `?:` (5.16), as the operand of `typeid`, `noexcept`, or `decltype`, as the expression in a return statement (6.6.3) for a function with the return type `cv void`, or as the operand of an explicit conversion to type `cv void`.~~

Change in §3.9.2 Compound types [basic.compound] paragraph 1 (remove redundancy now that void is an object type):

Compound types can be constructed in the following ways:

- *arrays* of objects of a given type, 8.3.4;
- *functions*, which have parameters of given types and return ~~void or~~ references or objects of a given type, 8.3.5;
- *pointers* to ~~cv void or~~ objects or functions (including static members of classes) of a given type, 8.3.1;
- *references* to objects or functions of a given type, 8.3.2. There are two types of references:
  - *lvalue reference*
  - *rvalue reference*
- *classes* containing a sequence of objects of various types (Clause 9), a set of types, enumerations and functions for manipulating these objects (9.3), and a set of restrictions on the access to these entities (Clause 11);
- *unions*, which are classes capable of containing objects of different types at different times, 9.5;
- *enumerations*, which comprise a set of named constant values. Each distinct enumeration constitutes a different *enumerated type*, 7.2;
- *pointers to non-static* <sup>52</sup> *class members*, which identify members of a given type within objects of a given class, 8.3.3.

Change in the first part of §3.9.2 Compound types [basic.compound] paragraph 3 (a pointer to void is now a pointer to object type):

The type of ~~a pointer to cv void or~~ a pointer to an object type is called an *object pointer type*. ~~[ Note: A pointer to void does not have a pointer-to-object type, however, because void is not an object type. — end note ]~~ ...

Change in §3.9.3 CV-qualifiers [basic.type.qualifier] paragraph 1 (remove redundancy now that void is an object type):

A type mentioned in 3.9.1 and 3.9.2 is a *cv-unqualified type*. Each type which is a cv-unqualified complete or incomplete object type ~~or is void (3.9)~~ has three corresponding cv-qualified versions of its type: a *const-qualified* version, a *volatile-qualified* version, and a *const-volatile-qualified* version. The term *object type* (1.8) includes the cv-qualifiers specified in the *decl-specifier-seq* (7.1), *declarator* (Clause 8), *type-id* (8.1), or *newtype-id* (5.3.4) when the object is created.

- A *const object* is an object of type `const T` or a non-mutable subobject of such an object.

– A *volatile object* is an object of type **volatile T**, a subobject of such an object, or a mutable subobject of a **const volatile** object.

– A *const volatile* object is an object of type **const volatile T**, a non-mutable subobject of such an object, a **const** subobject of a *volatile* object, or a non-mutable *volatile* subobject of a **const** object.

The cv-qualified or cv-unqualified versions of a type are distinct types; however, they shall have the same representation and alignment requirements (3.11).<sup>53</sup>

Change in §3.10 Lvalues and rvalues [basic.lval] paragraph 4 (remove redundancy now that void is complete type):

Unless otherwise indicated (5.2.2), prvalues shall always have complete types ~~or the void type~~; in addition to these types, glvalues can also have incomplete types. [ *Note*: class and array prvalues can have cv-qualified types; other prvalues always have cv-unqualified types. See Clause 5. – *end note* ]

Change in §5.2.2 Function call [expr.call] paragraph 3 (remove redundancy now that void is an object type):

If the *postfix-expression* designates a destructor (12.4), the type of the function call expression is **void**; otherwise, the type of the function call expression is the return type of the statically chosen function (i.e., ignoring the **virtual** keyword), even if the type of the function actually called is different. This return type shall be an object type, or a reference type ~~or cv-void~~.

Change in §5.2.3 Explicit type conversion (functional notation) [expr.type.conv] paragraph 2 (initialization now covered by value-initializing since void is an object type):

The expression **T()**, where **T** is a *simple-type-specifier* or *typename-specifier* for a non-array complete object type ~~or the (possibly cv-qualified) void type~~, creates a prvalue of the specified type, whose value is that produced by value-initializing (8.5) an object of type **T**; ~~no initialization is done for the void() case~~. [ *Note*: if **T** is a non-class type that is cv-qualified, the *cv-qualifiers* are discarded when determining the type of the resulting prvalue (Clause 5). – *end note* ]

Change in §5.3.1 Unary operators [expr.unary.op] paragraph 1 (remove redundancy now that void is a complete type):

The unary **\*** operator performs *indirection*: the expression to which it is applied shall be a pointer to an object type, or a pointer to a function type and the result is an lvalue referring to the object or function to which the expression points. If the type of the expression is “pointer to **T**”, the type of the result is “**T**”. [ *Note*: indirection through a pointer to an incomplete type ~~(other than cv-void)~~ is valid. The lvalue thus obtained can be used in limited ways (to initialize a reference, for example); this lvalue must not be converted to a prvalue, see 4.1. – *end note* ]

Change in §5.3.3 Sizeof [expr.sizeof] paragraph 1 (include void in explicitly mentioned types with implementation-defined size):

The **sizeof** operator yields the number of bytes in the object representation of its operand. The operand is either an expression, which is an unevaluated operand (Clause 5), or a parenthesized *type-id*. The **sizeof** operator shall not be applied to an expression that has function or incomplete type, to the parenthesized name of such types, or to a glvalue that designates a bit-field. **sizeof(char)**, **sizeof(signed char)** and **sizeof(unsigned char)** are 1. The result of **sizeof** applied to any other fundamental type (3.9.1) is implementation-defined. [ *Note*: in particular, **sizeof(void)**, **sizeof(bool)**, **sizeof(char16\_t)**, **sizeof(char32\_t)**, and **sizeof(wchar\_t)** are implementation-defined.<sup>75</sup> – *end note* ] [ *Note*: See 1.7 for the definition of *byte* and 3.9 for the definition of *object representation*. – *end note* ]

Remove §5.3.5 Delete [expr.delete] footnote 80 (void is now an object type and can be deleted):

~~This implies that an object cannot be deleted using a pointer of type **void\*** because **void** is not an object type.~~

Change in §5.9 Relational operators [expr.rel] paragraph 1 (void objects can be used with relational operators):

The relational operators group left-to-right. [ *Example*: **a<b<c** means **(a<b)<c** and not **(a<b)&&(b<c)**. – *end example* ]

```
relational-expression:
    shift-expression
    relational-expression < shift-expression
    relational-expression > shift-expression
    relational-expression <= shift-expression
    relational-expression >= shift-expression
```

The operands shall have arithmetic, enumeration, **void**, or pointer type. The operators < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to) all yield **false** or **true**. The type of the result is **bool**.

Change in §5.10 Equality operators [expr.eq] paragraph 1 (void objects can be used with equality operators):

The == (equal to) and the != (not equal to) operators group left-to-right. The operands shall have arithmetic, enumeration, **void**, pointer, or pointer to member type, or type **std::nullptr\_t**. The operators == and != both yield **true** or **false**, i.e., a result of type **bool**. In each case below, the operands shall have the same type after the specified conversions have been applied.

Add a new paragraph to §5.10 Equality operators [expr.eq] (specify equality for all instances of void):

Two operands of type **void** compare equal.

Change in §5.16 Conditional operator [expr.cond] paragraph 2 (regarding the result of the conditional operator):

If either the second or the third operand ~~has type **void** is a (possibly parenthesized) throw-expression (5.17)~~, one of the following shall hold:

- (2.1) – The second or the third operand (but not both) is a (possibly parenthesized) *throw-expression* (5.17); the result is of the type and value category of the other. The *conditional-expression* is a bit-field if that operand is a bit-field.
- (2.2) – Both the second and the third operands ~~have type **void** are (possibly parenthesized) throw-expressions~~; the result is of type **void** and is a prvalue. ~~[ Note: This includes the case where both operands are throw-expressions. – end note ]~~

Change in §6.6.3 The return statement [stmt.return] paragraph 2 (allow void expressions to be used in return statements in the same manner as other object types, and update wording in order to retain well-defined behavior when flowing off the end of a function with a void return type):

The *expr-or-braced-init-list* of a return statement is called its operand. A return statement with no operand shall be used only in a function whose return type is **cv void**, a constructor (12.1), or a destructor (12.4). ~~A return statement with an operand of type **void** shall be used only in a function whose return type is **cv void**. A return statement with any other operand shall be used only in a function whose return type is not **cv void**; the~~A return statement initializes the object or reference to be returned by copy-initialization (8.5) from the operand. [ Note: A return statement can involve the construction and copy or move of a temporary object (12.2). A copy or move operation associated with a return statement may be elided or considered as an rvalue for the purpose of overload resolution in selecting a constructor (12.8). – end note ] [Example:

```
std::pair<std::string,int> f(const char* p, int x) {  
    return {p,x};  
}
```

– end example ] Flowing off the end of a function that does not have the return type **cv void** is equivalent to a **return** with no value; this results in undefined behavior in a value-returning function. Flowing off the end of a function with the return type **cv void** is equivalent to that function returning a value-initialized **void**.

Change in §8.3.2 References [dcl.ref] paragraph 1 (allow references to void types).

Remove the final statement starting after the end of the last note:

~~... A declarator that specifies the type "reference to **cv void**" is ill-formed.~~

Change in §8.3.3 Pointers to members [dcl.mptr] paragraph 3 (allow pointer to void member types):

A pointer to member shall not point to a static member of a class (9.4), or a member with reference type, ~~or~~ **"cv void"**.

[ Note: See also 5.3 and 5.5. The type "pointer to member" is distinct from the type "pointer", that is, a pointer to member is declared only by the pointer to member declarator syntax, and never by the pointer declarator syntax. There is no "reference-to-member" type in C++. – end note ]

Change in the middle of §8.3.4 Arrays [dcl.array] paragraph 1 (allow arrays of void):

... *T* is called the array *element type*; this type shall not be a reference type, ~~the (possibly cv-qualified) type `void`~~, a function type or an abstract class type. ...

Change in §8.3.4 Arrays [**dcl.array**] paragraph 2 (allow arrays of void):

An array can be constructed from one of the fundamental types ~~(except `void`)~~, from a pointer, from a pointer to member, from a class, from an enumeration type, or from another array.

Change in §8.3.5 Functions [**dcl.fct**] paragraph 4 (allow void to be used as a function parameter type):

The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [ *Note*: the *parameter-declaration-clause* is used to convert the arguments specified on the function call; see 5.2.2. – *end note* ] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type `void` is equivalent to an empty parameter list. ~~Except for this special case, a parameter shall not have type cv void.~~ This is the only case for which a parameter of type cv void has special meaning.

If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack (14.5.3), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where “...” is not part of an *abstract-declarator*, “, ...” is synonymous with “...”. [ *Example*: the declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a `const char*` – *end example* ]

[ *Note*: The standard header `<cstdarg>` contains a mechanism for accessing arguments passed using the ellipsis (see 5.2.2 and 18.10). – *end note* ]

Change in the first part of §9.4.2 Static data members [**class.static.data**] paragraph 2 (void is no longer incomplete and may be a static data member):

The declaration of a **static** data member in its class definition is not a definition and may be of an incomplete type ~~other than cv-qualified void~~...

Change in §13.6 Built-in operators [**over.built**] paragraph 15 (built-in relational operator declarations are now provided for void):

For every *T*, where *T* is an enumeration type, `void` or a pointer type, there exist candidate operator functions of the form

```
bool operator<(T, T );
bool operator>(T, T );
bool operator<=(T, T );
bool operator>=(T, T );
bool operator==(T, T );
bool operator!=(T, T );
```

Change in §14.1 Template parameters [**temp.param**] paragraph 7 (allow void as a non-type template parameter type):

A non-type *template-parameter* shall not be declared to have floating point, or class, ~~or void~~ type. [ *Example*:

```
template<double d> class X;      // error
template<double* pd> class Y;    // OK
template<double& rd> class Z;    // OK
```

– *end example* ]

Change in §14.8.2 Template argument deduction [**temp.deduct**] paragraph 8.2 (details regarding when type deduction can fail):

– Attempting to create an array with an element type that is `void`, a function type, a reference type, or an abstract class type, or attempting to create an array with a size that is zero or negative. [ *Example*:

```
template <class T> int f(T[5]);
```



```
int I = f<int>(0);
int j = f<void>(0); // invalid array
int j = f<int&>(0); // invalid array

- end example ]
```

Remove **§14.8.2 Template argument deduction [temp.deduct]** paragraph 8.6 (details regarding when type deduction can fail):

~~- Attempting to create a reference to **void**.~~

Change in **§14.8.2 Template argument deduction [temp.deduct]** paragraph 8.10 (details regarding when type deduction can fail):

~~- Attempting to create a function type in which a parameter has a type of **void**, or in which the return type is a function type or array type.~~

Change in **§15.1 Throwing an exception [except.throw]** paragraph 3 (void is no longer incomplete and can be thrown):

Throwing an exception copy-initializes (8.5, 12.8) a temporary object, called the *exception object*. The temporary is an lvalue and is used to initialize the variable declared in the matching *handler* (15.3). If the type of the exception object would be an incomplete type or a pointer to an incomplete type ~~other than (possibly cv-qualified) **void**~~ the program is ill-formed.

Change in **§15.3 Handling an exception [except.handle]** paragraph 1 (void is no longer incomplete):

The *exception-declaration* in a *handler* describes the type(s) of exceptions that can cause that *handler* to be entered. The *exception-declaration* shall not denote an incomplete type, an abstract class type, or an rvalue reference type. The *exception-declaration* shall not denote a pointer or reference to an incomplete type, ~~other than **void\***, **const void\***, **volatile void\***, or **const volatile void\***.~~

Change in **§15.4 Exception specifications [except.handle]** paragraph 2 (void is no longer incomplete):

A type denoted in a *dynamic-exception-specification* shall not denote an incomplete type or an rvalue reference type. A type denoted in a *dynamic-exception-specification* shall not denote a pointer or reference to an incomplete type, ~~other than "pointer to cv **void**".~~ A type cv T denoted in a *dynamic-exception-specification* is adjusted to type T. A type "array of T", or function type T denoted in a *dynamic-exception-specification* is adjusted to type "pointer to T". A *dynamic-exception-specification* denotes an exception specification that is the set of adjusted types specified thereby.

Change in **§20.10.4.3 Type properties [meta.unary.prop]** (remove explicit mentions of void other than in is\_void):  
Simple, but too verbose to list explicitly in this paper, with changes spanning across multiple tables

Change in **C.4 C++ and ISO C++ 2014 [diff.cpp14]** (add a paragraph regarding changes made to the void type):

**Change:** void is a complete object type.  
**Rationale:** Makes the type system more consistent, and makes void easier to work with in generic code.  
**Effect on original feature:** Certain types and expressions involving void during substitution of template arguments may cause substitution to fail in C++ 2014 where substitution would succeed for object types, such as if a dependent void type appears as the operand of sizeof or is used as function parameter type. These uses of void will no longer cause substitution to fail.  
**Difficulty of converting:** When using a dependent void type as a means to exploit substitution failure for the purpose of ruling out specializations, alternative means to cause substitution to fail should be employed.

## Change History

This proposal is a revision of P0146r0<sup>[11]</sup>, which was a part of the September 2015 mailing. Changes that this proposal adds include a high-level overview section, more rationale, references to existing languages that offer an instantiable void type, more analysis of alternatives, and a compatibility section to **[diff.cpp14]**. It also no longer suggests deprecating the existing usage of T(void) as a means to specify a nullary function, due to concerns that we'd never actually be able to remove that meaning in a future standard, particularly due to C compatibility. This update also fixes the oversight of not fully specifying initialization of void now that it is an object type, and it also fixes the oversight of not explicitly defining the behavior when flowing off of the end of a function that returns a void type without an explicit return statement (it is now explicitly defined to be valid and produces a value-initialized void).

Changes to the standard have now also been updated to be relative to N4567<sup>[10]</sup>.

## Acknowledgments

Though a change to the language such as this has been discussed informally by several in the past, particularly by Sean Parent, the direct motivation for this proposal was a discussion<sup>[12]</sup> in the std-proposals group (a thread that the author of this proposal did not start). Many suggestions and alternatives mentioned in this proposal come from those who took part in that discussion.

In addition to those mentioned above, Richard Smith provided support and early feedback during the writing of this proposal and its revision, and provides continued aid in the specification of wording. I also hijacked his formatting for use in this document. Also thanks to Sean Parent and David Sankel, who provided feedback on drafts of this revision.

## References

- [1] Vicente J. Botet Escriba and Pierre Talbot: "A proposal to add a utility class to represent expected monad" N4109 <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4109.pdf>
- [2] Matt Calabrese: "Remove Future-Related Explicit Specializations for Void" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0241r0.html>
- [3] Matt Calabrese: "Standard Library Support for Void" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0242r0.html>
- [4] James C. Dehnert and Alexander Stepanov: "Fundamentals of Generic Programming" <http://www.stepanovpapers.com/DeSt98.pdf>
- [5] Apple Inc.: "The Swift Programming Language" [https://developer.apple.com/library/ios/documentation/Swift/Conceptual/Swift\\_Programming\\_Language/](https://developer.apple.com/library/ios/documentation/Swift/Conceptual/Swift_Programming_Language/)
- [6] The Rust Project Developers: "The Rust Reference" <https://doc.rust-lang.org/reference.html>
- [7] *Various Authors (wiki)*: "Haskell Language and Library Specification" [https://wiki.haskell.org/Language\\_and\\_library\\_specification](https://wiki.haskell.org/Language_and_library_specification)
- [8] Python Software Foundation: "The Python Language Reference" <https://docs.python.org/3/reference/index.html>
- [9] Ville Voutilainen: "constexpr\_if" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0128r0.html>
- [10] "Working Draft, Standard for Programming Language C++" N4567 <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf>
- [11] Matt Calabrese: "Regular Void" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0146r0.html>
- [12] *Various Participants*: "Allow values of void" Discussion <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/05prNzycvYU>